

Interactive Astronomy
Using 3D Computer Graphics
(MA2288 UROPS)

Wu ChengYuan

Supervisor: Associate Professor Helmer Aslaksen

Abstract

Understanding spherical astronomy requires good spatial visualization. Unfortunately, it is very hard to make good three dimensional (3D) illustrations and many illustrations from standard textbooks are in fact incorrect. There are many programs that can be used to create illustrations, but in this report we have focused on T_EX-friendly, free programs. We have compared MetaPost, PSTricks, Asymptote and Sketch by creating a series of illustrations related to the problem of why sundials sometimes go backwards in the tropics.

Contents

1	Introduction	3
1.1	Advantages of programmatic drawing programs	4
1.2	Evaluation criteria	4
2	MetaPost	7
2.1	Input/Output	8
2.2	3D	9
2.3	Dashing of hidden lines	9
2.4	Computation of intersections	9
2.5	Usability	10
2.6	Efficiency of code	10
2.7	Speed	10
2.8	Drawing of cones and other 3D objects	10
2.9	Code references	10
2.10	Diagrams	11
3	Asymptote	12
3.1	Input/Output	12
3.2	3D	13
3.3	Dashing of hidden lines	13
3.4	Computation of intersections	14
3.5	Usability	14
3.6	Efficiency of code	14
3.7	Speed	15
3.8	Drawing of cones and other 3D objects	15
3.9	Code references	15
3.10	Diagrams	16
4	PSTricks	21
4.1	Input/Output	21
4.2	3D	22

4.3	Dashing of hidden lines	22
4.4	Computation of intersections	23
4.5	Usability	23
4.6	Efficiency of code	24
4.7	Speed	24
4.8	Drawing of cones and other 3D objects	24
4.9	Code references	25
4.10	Diagrams	25
5	Sketch	29
5.1	Input/Output	29
5.2	3D	30
5.3	Dashing of hidden lines	30
5.4	Computation of intersections	30
5.5	Usability	30
5.6	Efficiency of code	31
5.7	Speed	31
5.8	Drawing of cones and other 3D objects	31
5.9	Code references	31
5.10	Diagrams	32
6	Conclusion	34
6.1	MetaPost	34
6.2	Asymptote	35
6.3	PSTricks	36
6.4	Sketch	37
6.5	Comparison	37
	Bibliography	39
	Acknowledgements	41

Chapter 1

Introduction

Understanding spherical astronomy requires good spatial visualization. It is therefore very helpful to be able to draw good illustrations. Unfortunately, it is very hard to make good three dimensional (3D) illustrations and many illustrations from standard textbooks are in fact incorrect [1].

In addition, when drawing the celestial sphere, it is necessary to cheat in some ways, since on one hand the Earth is supposed to be a small dot in the middle of the sphere, while at the same time we are supposed to be able to see the gnomon (the stick of the sundial) and the path of the shadow.

In this project we will use various programs to draw pictures related to spherical astronomy. We will use them inside T_EX documents, and we want to be able to recycle the code to see how the pictures depend on various parameters, so we have not considered free hand drawing programs, but only programmatic drawing programs. We have also not used computer algebra software (CAS). There are several such programs that could easily have created these pictures, but we decide to not include them in this round. One reason being that the most popular CAS programs are commercial.

We decided to use the following four programs:

1. MetaPost
2. Asymptote
3. PSTricks (using the pst-solides3d package)
4. Sketch

We would also have liked to test PGF/TikZ, but we ran out of time. We would like to include that at a later stage.

Our intention was not to present a comprehensive survey of the 3D capabilities of these programs. If we were to do a fair comparison, we would

have had to consider a wide range of test pictures. Instead we just picked pictures that we needed to draw for our own purposes.

We are not experts, and our goal is simply to show what two newbies can create in two months time. Experts could obviously have done a much better job. However, we hope that some of our experiences can be of interest to other people.

All the four programs are excellent, and we would like to thank all the people who have helped create them.

1.1 Advantages of programmatic drawing programs

There are many advantages of using programmatic drawing programs. Firstly, recycling and portability. For instance, to change the point of view, we just have to change a few parameters in the code, as opposed to drawing a new diagram from scratch.

Secondly, programmatic drawing programs are definitely more accurate than drawing by hand or using ordinary graphical tools like MS Paint. This is vital to produce accurate diagrams for books and other printed publications.

1.2 Evaluation criteria

We shall endeavour to compare the programs using the following criteria:

- 3D (is the program inherently 3D or 2D?)
- Dashing of hidden lines
- Computation of intersections
- Usability (ease of installation/usage/learning the language)
- Efficiency of code (measured by approximate length of code required)
- Speed (measured by compile time)
- Drawing of cones and other 3D objects

The first diagram we wish to draw using the four programs is shown in Figure 1.1. This diagram illustrates the equator and the ecliptic. It also draws the poles and the meridians. The inclination of the lunar orbit has intentionally been magnified.

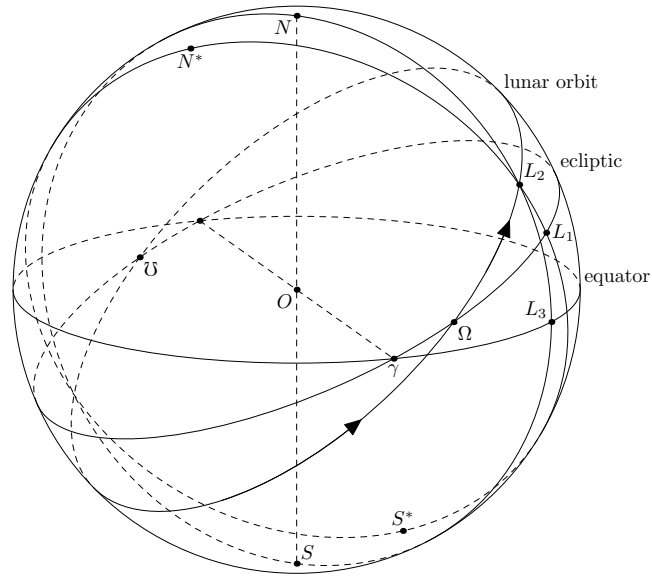


Figure 1.1: Denis Roegel, TUGboat, Vol. 30 (2009), No. 1 pg. 85 (drawn using MetaPost)

The second diagram we wish to draw is a picture from an honours thesis [10]. The diagram (Figure 1.2) shows the solar path and the shadow of a gnomon in the tropics.

Other than the above two pictures, we will also include other pictures for the reader's reference.

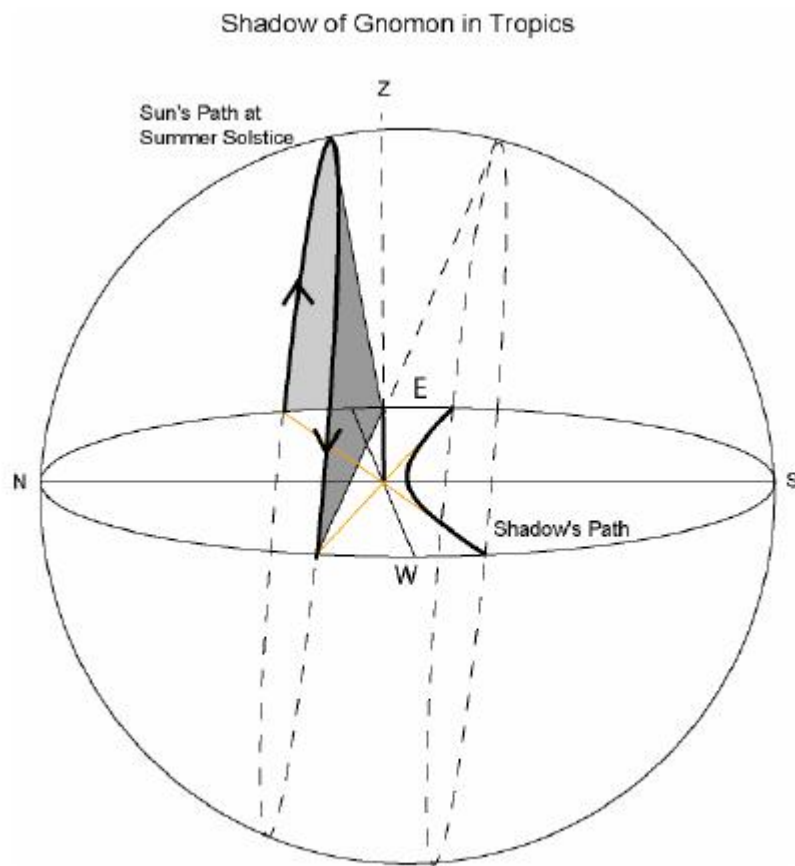


Figure 1.2: Diagram from the honours thesis by Lee, Sim and Yang

Chapter 2

MetaPost

MetaPost is described as follows on Wikipedia.

MetaPost refers to both a programming language and the interpreter of the MetaPost programming language. Both are derived from Donald Knuth's Metafont language and interpreter. MetaPost excels at producing diagrams in the PostScript programming language from a geometric/algebraic description. The language shares Metafont's elegant declarative syntax for manipulating lines, curves, points and geometric transformations [2].

The Metafont interpreter was written by Donald Knuth under an open source license, allowing John D. Hobby to adapt the interpreter to his own ends, giving us MetaPost.

For this project, our starting point was to closely examine a nicely drawn MetaPost picture by Denis Roegel [1]. Part of the code was included in the article, and some of the code was kindly given to us by the author. We started our project by trying to reproduce this picture using the other programs. His approach is to use the method of orthogonal projections to project a three-dimensional vector onto a plane. Before performing the projection, the sphere is oriented in space.

For the figures created by Roegel, the values $\theta = 70$ and $\phi = -15$ were chosen, where θ is the angle by which $\vec{i}$ is rotated around $\vec{k}$ and ϕ is the angle by which $\vec{k}$ is rotated around the rotated $\vec{i}$ axis.

The result is that the sphere is oriented in space as shown in Fig. 2.1.

This corresponds to a viewpoint of $(1, -0.342, 0.259)$.

$$\cos(70^\circ) = 0.342 \tag{2.1}$$

$$\sin(-15^\circ) = -0.259 \tag{2.2}$$

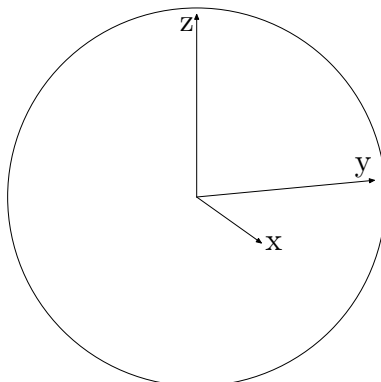


Figure 2.1: $\theta = 70$ and $\phi = -15$

2.1 Input/Output

There are various ways of including MetaPost code or figures into documents.

1. We can include MetaPost figures in $\text{\LaTeX}$ by using the `graphicx` package and then the `\includegraphics` command. After that, we need to declare that we are using MetaPost figures in $\text{\LaTeX}$:
`\DeclareGraphicsRule{*}{mps}{*}{}.`

The point of `\DeclareGraphicsRule{*}{mps}{*}{}.` is to tell pdf $\text{\LaTeX}$ that unknown file extensions (in this case numbers) should be treated as MPS files.

2. We can rename the MetaPost figures manually, giving them a `.mps` extension. This way, we do not need to use the `\DeclareGraphicsRule` command.
3. We can tell MetaPost to use `.mps` extensions for figures that it outputs. The way to do this is to define the `outputtemplate` in the MetaPost code itself [3, pg. 5].
4. We can embed MetaPost code directly in $\text{\LaTeX}$ documents by using the `EMP` package.

MetaPost produces encapsulated postscript output (EPS). The EPS files produced by MetaPost typically have names like `example.1`, `example.2`. MetaPost's PostScript output is a low-featured dialect of the Postscript language, called purified EPS, that can be converted into the Portable Document

Format (PDF) language on-the-fly. For that reason, MetaPost graphics can be handled by both engines, the traditional $\text{\TeX}$ with an external output driver and the newer $\text{pdf\TeX}$, that contains a built-in PDF output driver [3, pg. 6].

2.2 3D

MetaPost is inherently 2D in nature, and does not have any 3D vector type. However, a clever workaround is to use its `color` type with three numerical components which can be disguised as a vector [1].

Functions like the dot product, vector product, projection, rotation of vectors can then be defined.

2.3 Dashing of hidden lines

In MetaPost, there is no simple way of automatically dashing the hidden lines. In his paper, Roegel only discussed the case of determining the limits of visibility of the parallels to the equator [1, pg. 86]. The limits of visibility are determined by the intersection between the plane orthogonal to the viewing direction, and the circle representing the parallel. This intersection can consist in zero points (the parallel is then totally visible, or totally hidden), two points (there is both a hidden and a visible part), or one point (this is the limit case between the two previous cases).

Once the intersections are obtained in space, the two arcs are drawn separately. The hidden arc is drawn with dashed lines, while the visible arc is drawn with solid lines.

2.4 Computation of intersections

MetaPost has two built-in operators for calculation of points of intersections, namely `intersectionpoint` and `intersectiontimes`. These two operators are essentially 2D, but are still useful as we can find the points of intersection after projecting the relevant curves onto a 2D plane.

`intersectiontimes` gives the parametric locations on both paths, i.e., the values of the parameter `t`. `intersectionpoint`, as its name implies, gives the point of intersection between two paths.

2.5 Usability

MetaPost is fairly easy to use and learn. The main difficulty arises when trying to use MetaPost to draw 3D diagrams, as users will inevitably have to write their own code to deal with 3D vector projection.

2.6 Efficiency of code

To draw Fig. 1.1, approximately 443 lines of code are needed. A significant part of it (294 lines) are due to definitions and writing of macros to overcome the 3D limitations of MetaPost. The number of lines of code is just a rough estimate of the code efficiency, since it can vary in many ways by changing the layout and presentation of the code (writing a lot of code in one single line, for instance).

2.7 Speed

Compile time is no issue with MetaPost. It took less than a second to compile the .mp file for Figure 1.1 on a very ordinary Dell Inspiron I1420 laptop.

2.8 Drawing of cones and other 3D objects

MetaPost is inherently a 2D program, so in order to draw more general 3D objects like cones in MetaPost, we need to either design them from scratch, the way Roegel did it for spheres [1], or use a 3D MetaPost package like m3D by Anthony Phan [11]. However, using m3D to draw cones is not easy. We would need to use the `revolution` function defined in `m3Dplain.mp`.

The package m3D is still in progress, and the manual is still incomplete (only 12 pages). Hence, mastering the m3D package would only be possible for advanced MetaPost users who are able to learn directly from the `m3Dplain.mp` source code. Unfortunately, we did not have time to try this further.

2.9 Code references

For MetaPost, we referred to the TUGboat article by Roegel [1] and additional code obtained from Roegel through e-mail. The article is comprehensive, and we learnt a lot from it. We also referred to the MetaPost manual [3] to learn more about MetaPost syntax.

2.10 Diagrams

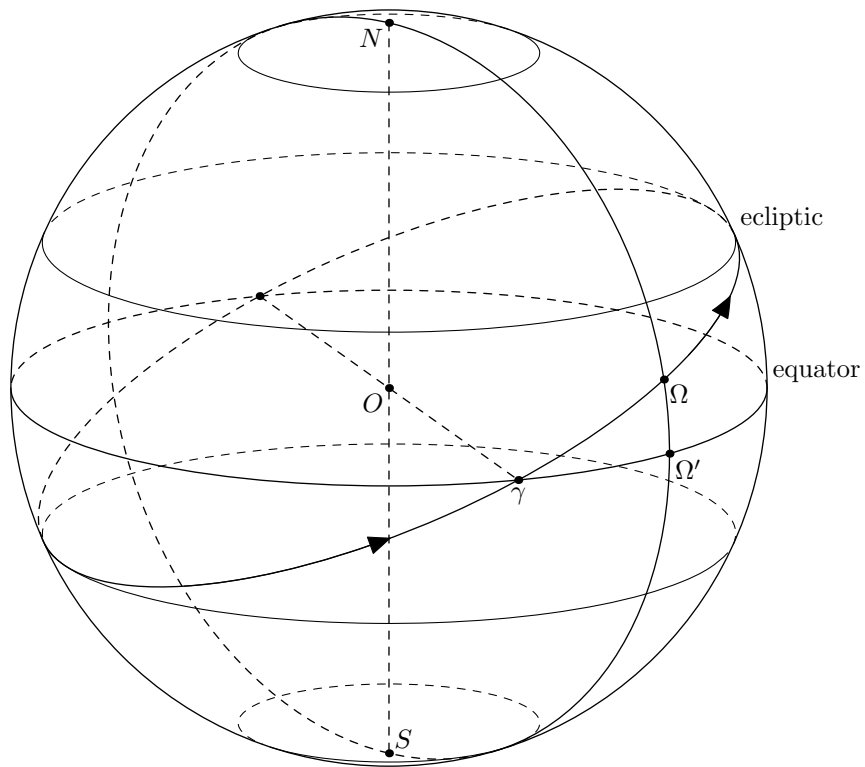


Figure 2.2: Denis Roegel, TUGboat, Vol. 30 (2009), No. 1 pg. 82

Chapter 3

Asymptote

Asymptote is described as follows on Wikipedia.

Asymptote is a descriptive vector graphics language, developed by Andy Hammerlindl, John C. Bowman (University of Alberta), and Tom Prince, that provides a natural coordinate-based framework for technical drawing. It is inspired by MetaPost, but has a C++-like syntax.

It is mathematically oriented (e.g. rotation of vectors by complex multiplication). Asymptote fully generalizes MetaPost path construction algorithms to three dimensions. It also appears to be the first software package to lift TeX into three dimensions [4].

3.1 Input/Output

There are several ways of inputting Asymptote code.

1. Drawing in batch mode: create a text file `test.asy` containing Asymptote code and compile using the Asymptote program. This method, known as batch mode, outputs a PostScript file `test.eps` [6, documentation section 10]. This EPS file can then be included in the TeX file.
2. Include Asymptote code in L^AT_EX documents. We use the Asymptote package: `\usepackage{asymptote}`, and we can include the Asymptote code itself within the `\begin{asy} ... \end{asy}` environment. The advantage of using this method is that we can tell Asymptote to output PDF files. For first-time users, this can prove to be tricky to set up, so we refer you to an excellent website for more details [13].

3. There is a third way, drawing in interactive mode, where Asymptote reads individual commands as they are entered by the user. Generally, this is not an efficient way to compile Asymptote code, as it involves line by line execution of commands.

As stated above, Asymptote can produce figures in either `.eps` or `.pdf` format. While using method two above to compile Asymptote code, we noticed that Asymptote produces three PDF files. One is in rotatable PRC (Product Representation Compact) format but uncropped, one is a normal cropped PDF, and the last one is a totally blank PDF with the same dimensions as the normal cropped PDF. We do not know why the totally blank PDF is generated.

While creating animations with Asymptote using the `animate` package, a `.js` file is created. This is used to create JavaScript (`js`) driven PDF animations.

3.2 3D

Asymptote is fully three dimensional, and can produce 3D PRC output. PRC (Product Representation Compact) is a 3D file format that can be used to embed 3D data in a PDF file.

Asymptote supports the data type `triple` which is an ordered triple of real components (x,y,z) used for three-dimensional drawings. This is a big advantage over MetaPost, which only had the `pair` data type.

Asymptote also supports the data type `path3` which is a 3D version of MetaPost's `path`. This is also very useful for drawing 3D diagrams. For instance, in our code we used:

```
path3 equator=circle(0,5,Z);
path3 ecliptic=rotate(23.5,X)*equator;
```

This draws the equator as a 3D circle centered at the origin, with radius 5 units, and normal vector pointing in the Z direction. The ecliptic is then defined as the equator rotated 23.5 degrees about the x-axis.

In our code, we used the `solids` geometry package to draw the sphere. This solid geometry package defines a `revolution` structure that can be used to fill and draw surfaces of revolution.

3.3 Dashing of hidden lines

In Asymptote, there is a structure `skeleton` which contains the three-dimensional wireframe used to visualize a volume of revolution. This can be used to draw

dashed lines for hidden parts of parallels and meridians with relative ease. For other circles, it is not as straightforward. Our code uses the following to draw dashed lines for the equator automatically:

```
skeleton s;
r.transverse(s,reltime(r.g,0.5));
draw(s.transverse.back,linetype('8 8',8));
draw(s.transverse.front);
```

3.4 Computation of intersections

Being inspired by MetaPost, Asymptote has the `intersectionpoints` operator which is the three-dimensional equivalent of that in MetaPost. For instance, in our code, to calculate the point of intersection between the ecliptic and the equator curves, we do the following:

```
triple[] eq_ec=intersectionpoints(ecliptic,equator,-1);
```

The value -1 indicates that the computations are performed to machine precision.

3.5 Usability

Asymptote is similar to MetaPost in many ways, and would be easy to learn for users familiar with MetaPost. There is also a wealth of predefined constructions in Asymptote, and for our code there was no need to write our own macros or functions. Asymptote also has an impressive collection of examples on the website by Philippe Ivaldi that users can learn from [5].

3.6 Efficiency of code

To draw Fig. 1.1, approximately 134 lines of code are needed. This is quite good, compared to the 443 lines needed for MetaPost. The main reason would definitely be the ready availability of predefined constructions for manipulating 3D vectors, so we did not have to write our own. For instance, one of the very useful built-in transform3 types is the `rotate` function:

```
rotate(real angle, triple v)
```

This rotates a three-dimensional object by `angle` in degrees about an axis v through the origin.

3.7 Speed

It took 1 min 3 s for Asymptote to compile Fig. 1.1. This is rather slow compared to the almost instantaneous output of MetaPost. This might prove to be a drawback when drawing very complicated figures.

3.8 Drawing of cones and other 3D objects

There are three ways of drawing cones in Asymptote. The first is to use Asymptote's default `cone` constructor. The constructor takes in various parameters such as the cone's height, radius of the cone's generating circle, coordinates of the vertex and the direction of the cone's axis.

```
cone(0,r,h,axis=Z)
```

The above code means that the cone's vertex is at the origin, its generating circle's radius is `r`, its height is `h` and its axis is the Z-axis.

The second way of drawing cones in Asymptote is to draw the cone parametrically. We can draw parametric surfaces for a function $f(u, v)$. The advantage of doing so is greater control since the boolean array or function `cond` can be used to control which surface mesh cells are actually drawn [6].

The third way is to draw cones using the `revolution` function defined in Asymptote. This is useful when trying to draw a cone by revolving a straight line around an axis.

3.9 Code references

To learn how to draw spheres and cones in Asymptote, we referred to examples on the website by Philippe Ivaldi [5]. We found that learning from examples is fast and efficient. The above-mentioned website is very extensive and contains numerous examples. The only problem is that the website is partially in French. We can search for relevant examples by using the “recherche” button found on the top right corner of the website.

Some of the examples we found useful are listed below.

- **Outline of sphere**

<http://www.piprime.fr/755/asymptote-using-solids-asy-fig0040/>

- **Parallels and meridians of a sphere**

<http://www.piprime.fr/754/asymptote-using-solids-asy-fig0030/>

- **Sphere with labels and straight lines**

http://www.piprime.fr/760/asymptote-using-solids_asy-fig0090/

- **Cone drawn using default constructor**

http://www.piprime.fr/995/official_asymptote_example-cones/

- **Cone drawn parametrically**

http://www.piprime.fr/1104/official_asymptote_example-parametricsurface/

- **Cone drawn using revolution**

http://www.piprime.fr/1190/official_asymptote_example-workcone/

3.10 Diagrams

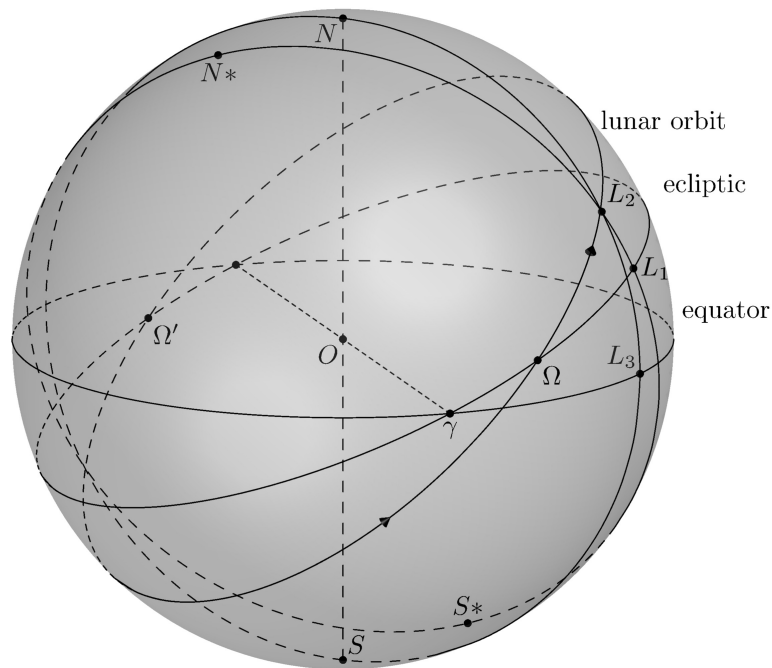


Figure 3.1: Figure 1.1 drawn using Asymptote

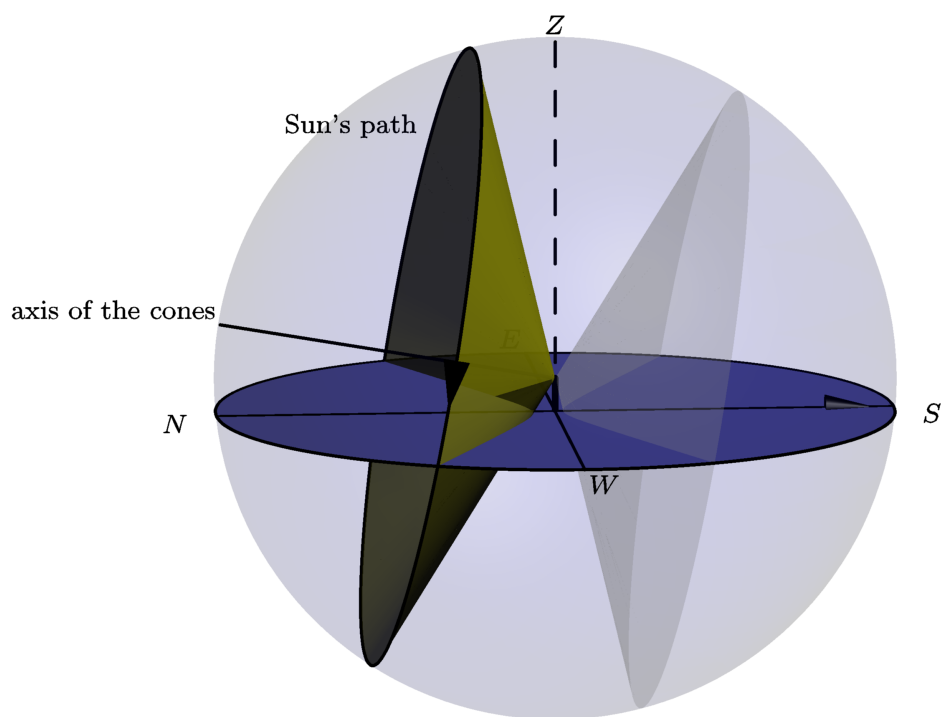


Figure 3.2: Figure 1.2 drawn using Asymptote

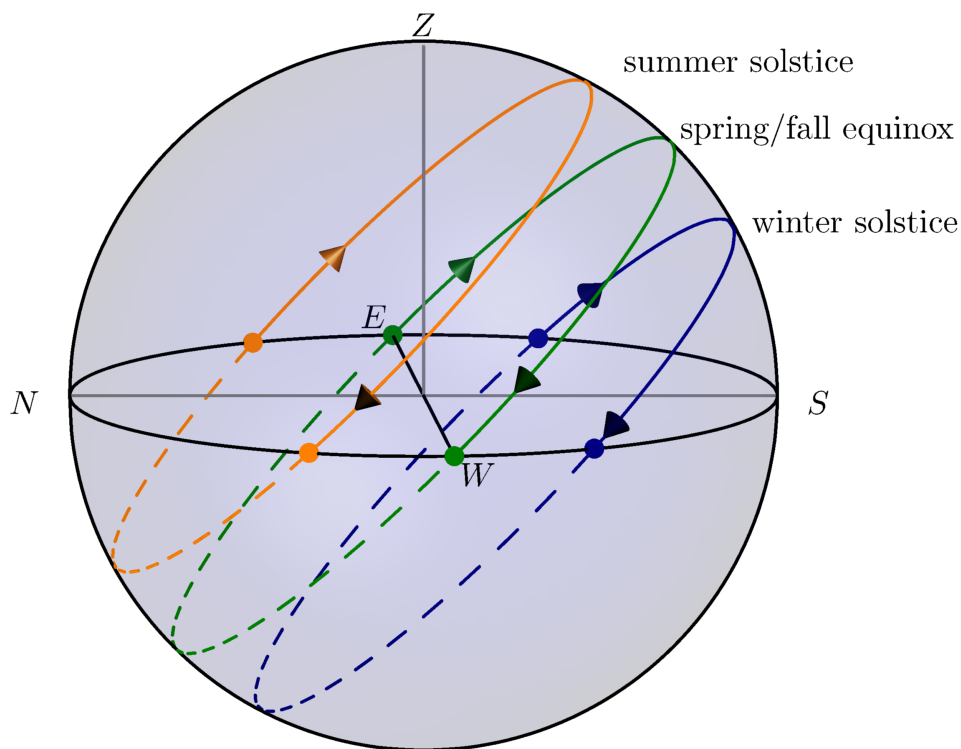


Figure 3.3: Paths of the Sun during the solstices and the equinoxes for an observer in the northern hemisphere

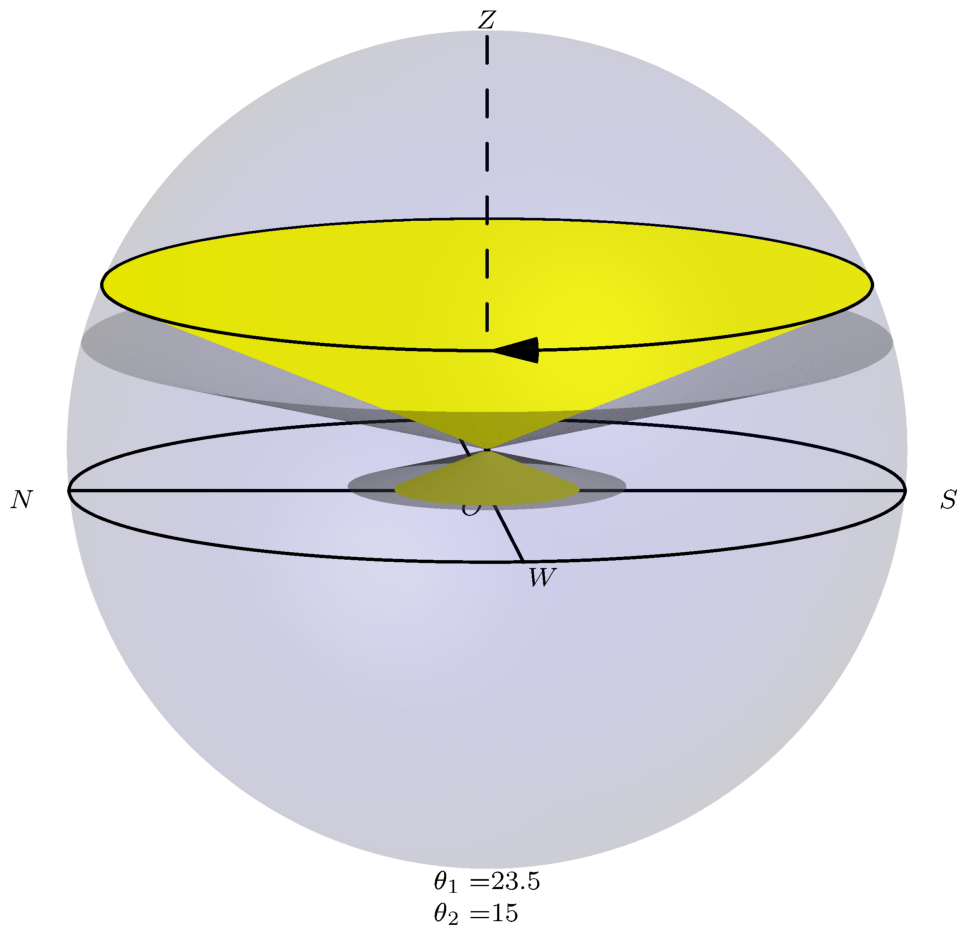


Figure 3.4: When the cones are flattened, we get a larger circle on the ground. The cones correspond to declinations of 23.5 and 15 degrees respectively.

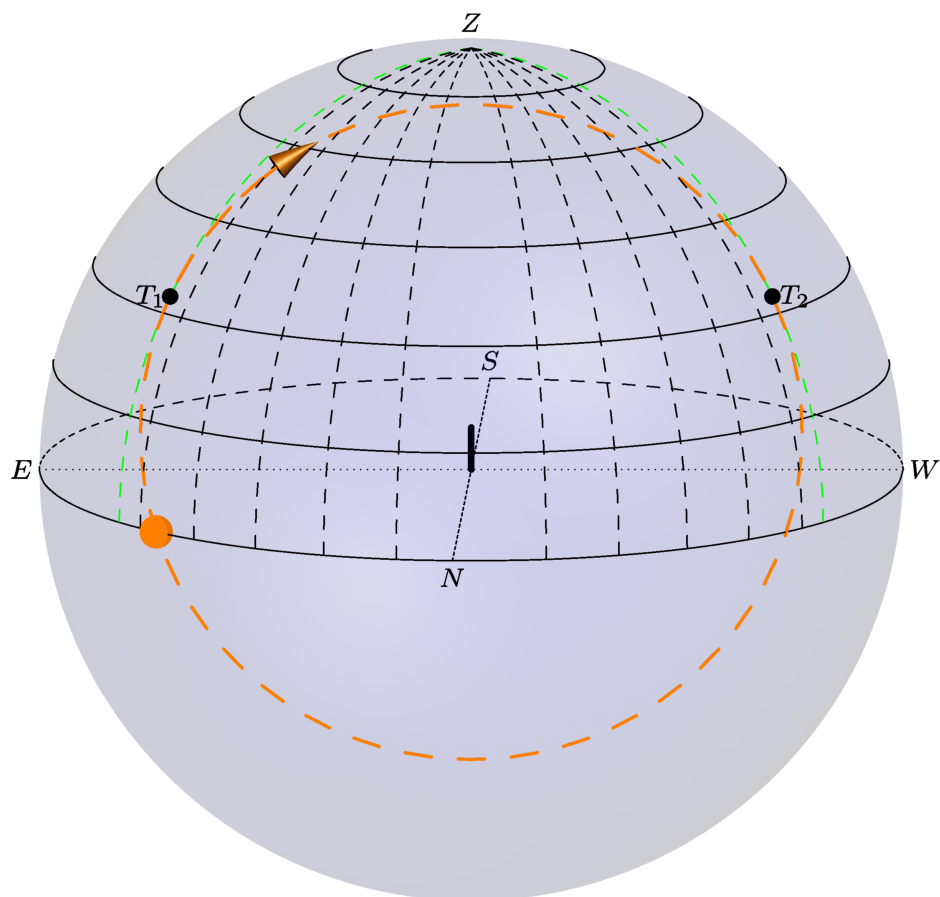


Figure 3.5: Daily path of the Sun in the northern part of the tropics on a day between the summer solstice and the zenith passage, showing how the Sun “turns” in the sky at points T_1 and T_2 .

Chapter 4

PSTricks

PSTricks is described as follows on Wikipedia.

PSTricks is a set of macros that allow the inclusion of PostScript drawings directly inside $\text{\TeX}$ or $\text{\LaTeX}$ code. It is originally the work of Professor Timothy Van Zandt and in recent years it has been maintained by Denis Girou, Sebastian Rahtz and Herbert Voss. ... PSTricks commands are low level, so many $\text{\LaTeX}$ packages have been made in order to ease the creation of several kinds of graphics that are commonly used on mathematical typesetting [7].

The PSTricks package `pst-solides3d` allows for 3D views of predefined or user-generated solids. Most of its functionality is accessible by way of three $\text{\TeX}$ macros: `\psSolid`, which can manipulate objects in 3 dimensions, `\psSurface`, related to the first macro and designed to represent surfaces that are defined by an equation of the type $f(x, y) = z$ and `\psProjection` which allows the user to project two-dimensional graphics/text onto any plane face of a 3D solid [8].

4.1 Input/Output

We can include PSTricks and `pst-solides3d` code in $\text{\LaTeX}$ by using the two packages.

```
\usepackage{pst-solides3d}
\usepackage{pstricks}
```

Then, we include our code within the `\begin{pspicture}... \end{pspicture}` environment.

PSTricks, as its name suggests, outputs PostScript by default. To output PDF, we can do the following [14, pg. 251]:

1. Use compilers with internal support of PSTricks. Such compilers include Bakoma $\text{\TeX}$ by Basil K. Malyshev and $\text{V}\text{\TeX}$ from MicroPress Inc..
2. Use external PostScript to PDF converters. The Acrobat software from Adobe, GhostScript from Aladdin Enterprises and PStill from Frank Siebert allow conversion of PostScript files to PDF.
3. Use Pdf $\text{\TeX}$ workarounds. The Pdf $\text{\TeX}$ compiler does not have a PostScript interpreter, and consequently cannot process EPS external files. There are several workarounds to this, but are too lengthy to be described here. We refer the reader to the PSTricks documentation [14, pg. 252].

4.2 3D

The package `pst-solides3d` is capable of drawing and viewing many 3D solids, including the sphere and the cone. The coordinates of the view point (V), can be set up in the same axes of coordinates, either in spherical coordinates with the additional option `[rtp2xyz]`, or in Cartesian coordinates which is the default option.

In our code, we set the viewpoint as follows:

```
\psset{viewpoint=100 -34.2 25.9,Decran=125}%
```

We obtained the numbers -34.2 and 25.9 by following the same choice $\theta = 70$ and $\phi = -15$ mentioned earlier in the MetaPost chapter. Due to the option `Decran` which is peculiar to PSTricks, we have to reverse the signs of the values to get the same viewpoint as that in MetaPost. We then multiply the values by hundred to adjust the diagram to a reasonable size, as the size of the diagram is also affected by the viewpoint.

4.3 Dashing of hidden lines

The package `pst-solides3d` provides a convenient way of dashing hidden lines automatically via the “Lines of intersecting planes” method [8]. To use this method, we need to know the equation $ax + by + cz + d = 0$ of the intersecting plane. We will then use `[a b c d]` as one of parameters in `\psSolid`.

In our code, we wrote it as follows:

```
\psSolid[object=sphere,
```

```
...
```

```
intersectionplan={[0 0 1 0]
```

```
...
```

`[0 0 1 0]` in this case represents the equation of the equator: $0x + 0y + 1z + 0 = 0$.

4.4 Computation of intersections

A nice and helpful option in `pst-solides3d` is `algebraic`, with which we can avoid RPN (Reverse Polish Notation).

However, `pst-solides3d` notably lacks the `intersectionpoint` operator that can be found in MetaPost and Asymptote. This makes it difficult, if not impossible, to calculate points of intersections using `pst-solides3d` alone.

A note in the documentation addresses the issue of calculation, albeit in the section on drawing tubes:

“Note: It is often advisable to calculate in advance, by hand or with a preferred software ...” [8, pg. 88]

In our code, we calculated the equation of the planes using Asymptote. For instance:

```
label(string(xpart(normal(ecliptic))), (0,8,2), E);
label(string(ypart(normal(ecliptic))), (0,8,1), E);
label(string(zpart(normal(ecliptic))), (0,8,0), E);
```

The above code displays the normal vector of the ecliptic plane, calculated by Asymptote. The code is displayed at a convenient location, for instance east (E) of the point (0,8,2). The function `string()` converts a real number to text so that it can be displayed as a `label`. The functions `xpart()`, `ypart()`, `zpart()` return the x , y , z components of a vector.

This will enable us to calculate the equation of the ecliptic plane, which we found to be $0x - 0.399y + 0.917z + 0 = 0$.

4.5 Usability

The `pst-solides3d` package is updated frequently and requires the latest versions of PSTricks and many other packages. Older versions may cause an error.

The `pst-solides3d` package has some French commands, so the user will have to know a bit of French in order to use the package. For our code, a French word we had to use was *noir*, which means *black*.

```
intersectioncolor=(noir)
```

Before we incorporated the “Lines of intersecting planes” method, we also had to use the command *courbe*, which means *curve*.

```
\psSolid[object=courbe,...]
```

4.6 Efficiency of code

To draw Fig. 1.1, approximately 99 lines of code are needed. This is much shorter compared to MetaPost’s 443 lines of code. It is also shorter than the 134 lines needed for Asymptote. However, note that points of intersection were not computed in the code.

4.7 Speed

It took 6 seconds for the `pst-solides3d` code to compile Fig. 1.1.

4.8 Drawing of cones and other 3D objects

We found that the easiest way to draw the cones is to draw them parametrically. The parameterised surfaces are setup as $[x(u, v), y(u, v), z(u, v)]$ and administered by the option `object=surfaceparametree` [8].

A right cone of height h and base radius r oriented along the z -axis, with vertex pointing up, and with the base located at $z = 0$ can be described by the parametric equations:

$$x = \frac{h - u}{h} r \cos \theta \quad (4.1)$$

$$y = \frac{h - u}{h} r \sin \theta \quad (4.2)$$

$$z = u \quad (4.3)$$

for $u \in [0, h]$ and $\theta \in [0, 2\pi]$.

To rotate the cone around the axes Ox , Oy and Oz , we can use the option `[RotX=valueX, RotY=valueY, RotZ=valueZ]` (angles in degrees). Alternatively, we can do the rotation to the parametric equations themselves by multiplying with a suitable 3D rotation matrix.

4.9 Code references

Compared to Asymptote, there are fewer examples for `pst-solides3d` code on the internet. Fortunately, the documentation [8] is detailed and contains useful examples. The documentation also includes a very short introduction to basic Reverse Polish Notation (RPN), which is the default syntax for calculations in PSTricks. It is not possible to avoid RPN completely even with the option `[algebraic]`, because some methods do not support the option.

Some of the examples we found useful are listed below.

- **Drawing spheres**

Documentation pg. 25

- **Drawing cones**

Documentation pg. 115

- **Drawing the intersection between a plane and a solid**

Documentation pg. 128

- **Dashing of hidden lines**

Documentation pg. 128

- **Reverse Polish Notation (RPN)**

Documentation pg. 117

4.10 Diagrams

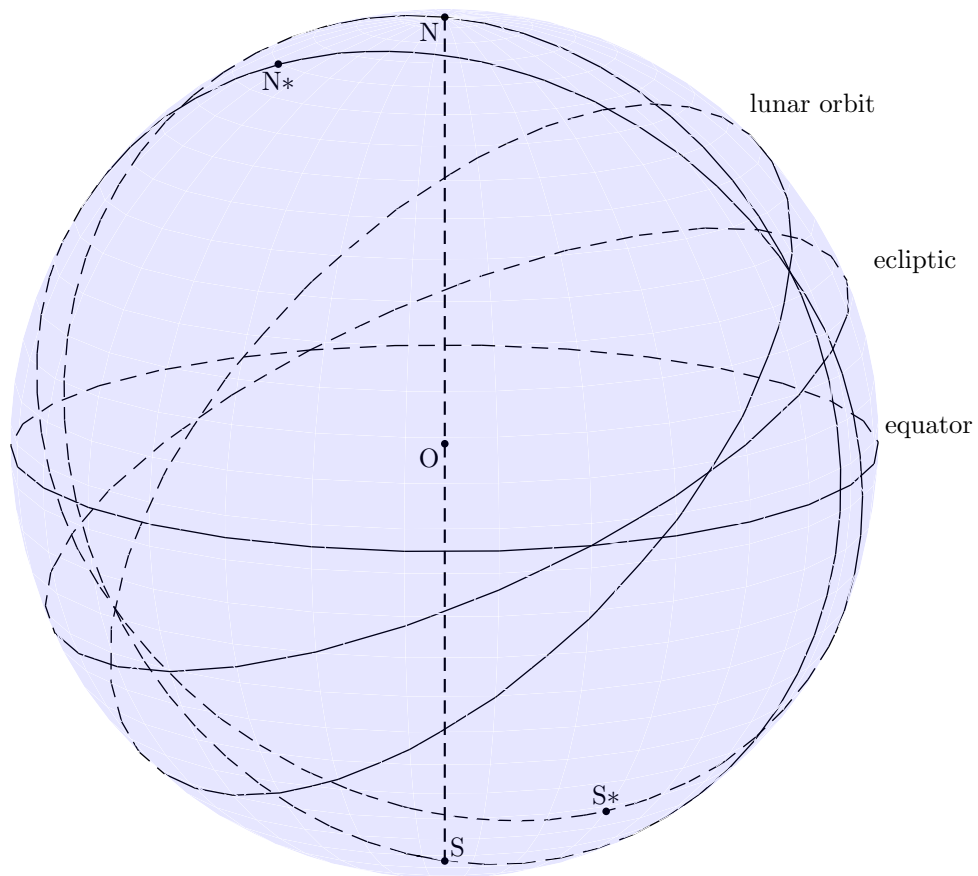


Figure 4.1: Figure 1.1 drawn using the PSTricks package pst-solides3d

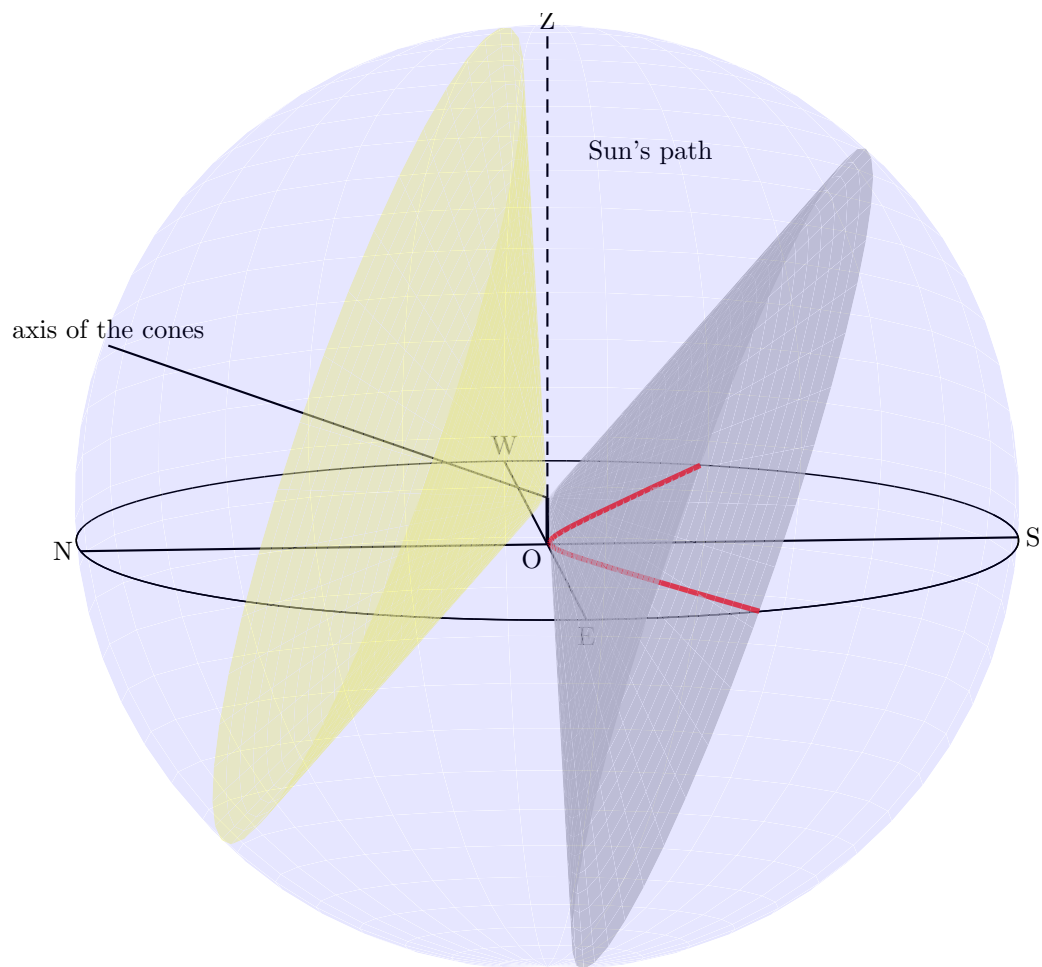


Figure 4.2: Figure 1.2 drawn using the PSTricks package pst-solides3d

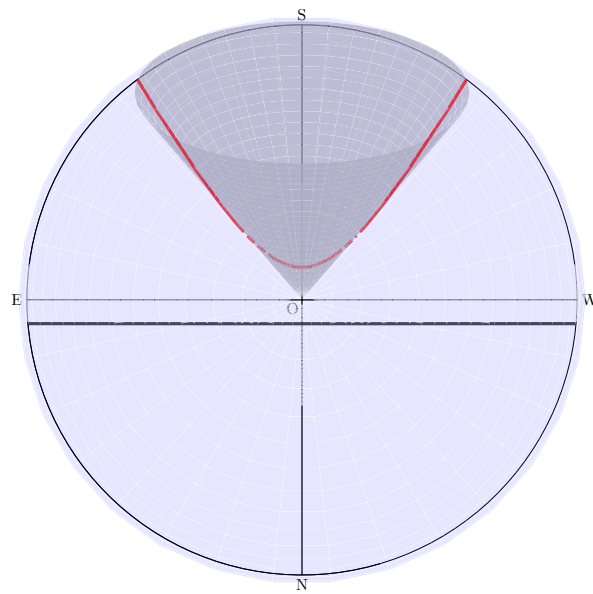


Figure 4.3: Diagram showing shadow cast by gnomon (red) and the equinoxial shadow line (black)

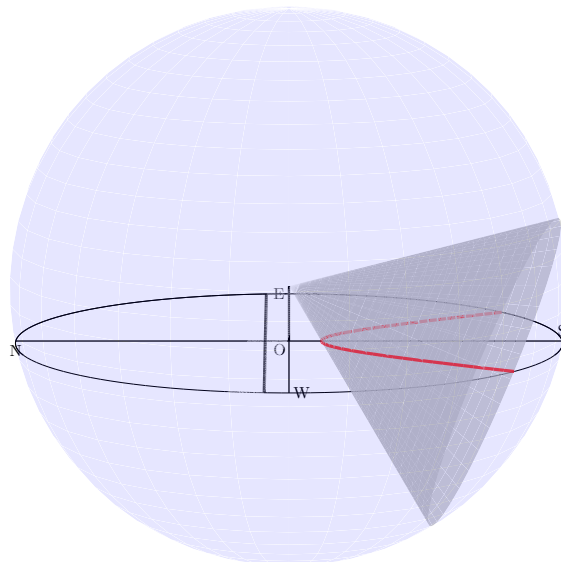


Figure 4.4: The previous diagram viewed from another viewpoint

Chapter 5

Sketch

Sketch, created by Eugene K. Ressler, is described as follows on its webpage.

The program Sketch is a small, simple system for producing line drawings of two- or three-dimensional solid objects and scenes. It began as a way to make illustrations for a textbook after the creators could find no suitable tool for this purpose. Existing scene processors emphasized GUIs and/or photo-realism, both un-useful to the creators. The creators of Sketch wanted to produce finely wrought, mathematically-based illustrations with no extraneous detail. The input language is reminiscent of PSTricks, so it will be easy to learn for current PSTricks users [9].

5.1 Input/Output

Sketch has to be run from the command line in Windows. For instance, we have to type something like:

```
sketch sketchcode.sk -o output.tex
```

Sketch will write PSTricks code to the file named `output.tex`. It is not possible to include Sketch code directly into $\text{\LaTeX}$ documents.

The PSTricks code Sketch produces is low-level code that is sometimes hard for humans to interpret. This is especially so when drawing curves, something that Sketch is not good at.

Sample PSTricks output:

```
\psline[linestyle=dashed](-1.571,1.094)(-1.483,1.101)
(-1.394,1.107)(-1.304,1.113)(-1.214,1.119)(-1.124,1.125)
(-1.033,1.129)(-.942,1.134)(-.85,1.138)(-.758,1.142)(-.666,1.145)
```

```
(-.573,1.148)(-.481,1.15)(-.388,1.152)(-.295,1.154)
(-.202,1.155)(-.109,1.156)(-.015,1.156)(.078,1.156)...
```

Other than producing PSTricks output, Sketch can also produce TikZ/PGF output.

5.2 3D

The direct transform constructor allows direct entry of a 4x4 transformation matrix. Most 3d graphics systems, including Sketch, use 4-dimensional homogeneous coordinates internally. These are transformed by multiplication with 4x4 matrices. The built-in constructors (`rotate`, `scale`, etc.) are all translated internally to such 4x4 matrices.

In particular, the `rotate(A,P,X)` constructor was very useful for our code. It rotates `A` degrees about point `P` with axis `X` according to the right hand rule.

5.3 Dashing of hidden lines

In Sketch, we did the dashing of hidden lines manually. Sketch has a “hidden surface algorithm”, and the manual shows an example of “an interesting trick that shows the hidden parts as dotted lines” [9]. However, we did not have time to try this on our examples.

5.4 Computation of intersections

Like `pst-solides3d`, Sketch unfortunately does not have the `intersectionpoint` operator or its equivalent. Hence it is difficult, if not impossible, to calculate points of intersection using Sketch alone.

5.5 Usability

The current version of Sketch has no built-in definition of a curve or a Bézier function. Curves are drawn using polygonal approximation. For our code, we approximated each circle by a 100-sided polygon:

```
def n_segs 100
def sphere {
sweep {n_segs, rotate(360 / n_segs, (0,0,0),[0,0,1]) } (1,0,0)
}
```

This code sweeps the point (1,0,0) 100 times by rotating it $360/100 = 3.6$ degrees each time and connecting the resulting points with line segments.

The end result is virtually indistinguishable from a smooth circle.

5.6 Efficiency of code

To draw Fig. 1.1, approximately 162 lines of code are needed.

5.7 Speed

Compilation of the Sketch code for Fig. 1.1 took less than a second. We only take into account the time for Sketch to convert Sketch code into PStricks code. The time taken for compilation of the PStricks code is also very fast, so it is negligible.

5.8 Drawing of cones and other 3D objects

In Sketch, a cone is drawn by rotating a line. “The cone shape is just a swept line with no closure tag and culling turned off” [9].

In our code, we drew the cone using the following:

```
def uppercone {
def p0 (0,0,0)
def p1 (0,coneradius,coneheight)
def seg_rot rotate(360/N,[Z])
sweep[cull=false,fillcolor=gray,linecolor=white,linewidth=0,opacity=0.5]
{N,[[seg_rot]]}line(p0)(p1)
}
```

This rotates the line connecting p0 and p1 about the Z-axis. The value N controls the number of polygons that the surface of the cone is made up of. (Recall that Sketch draws curves by polygonal approximation.) We set N to be 50.

5.9 Code references

There are few examples for Sketch online. We learnt from reading the Sketch documentation [9].

Some of the examples we found useful are listed below.

- **Drawing a circle**

<http://www.frontiernet.net/~eugene.ressler/sketch.html#Swept-objects>

- **Drawing a cone**

<http://www.frontiernet.net/~eugene.ressler/sketch.html#A-technical-drawing>

5.10 Diagrams

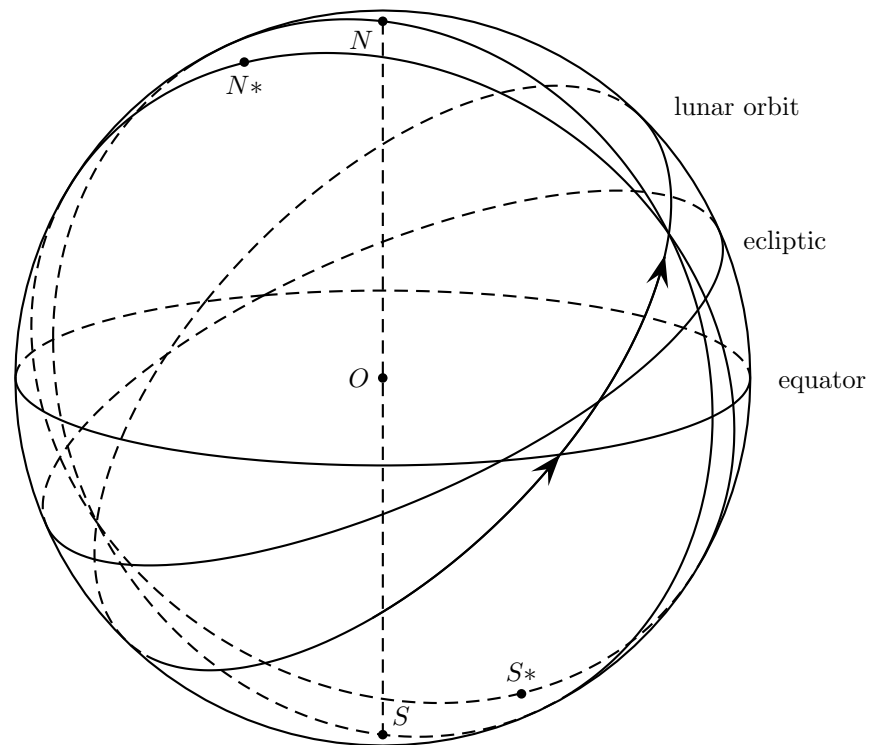


Figure 5.1: Figure 1.1 drawn using Sketch

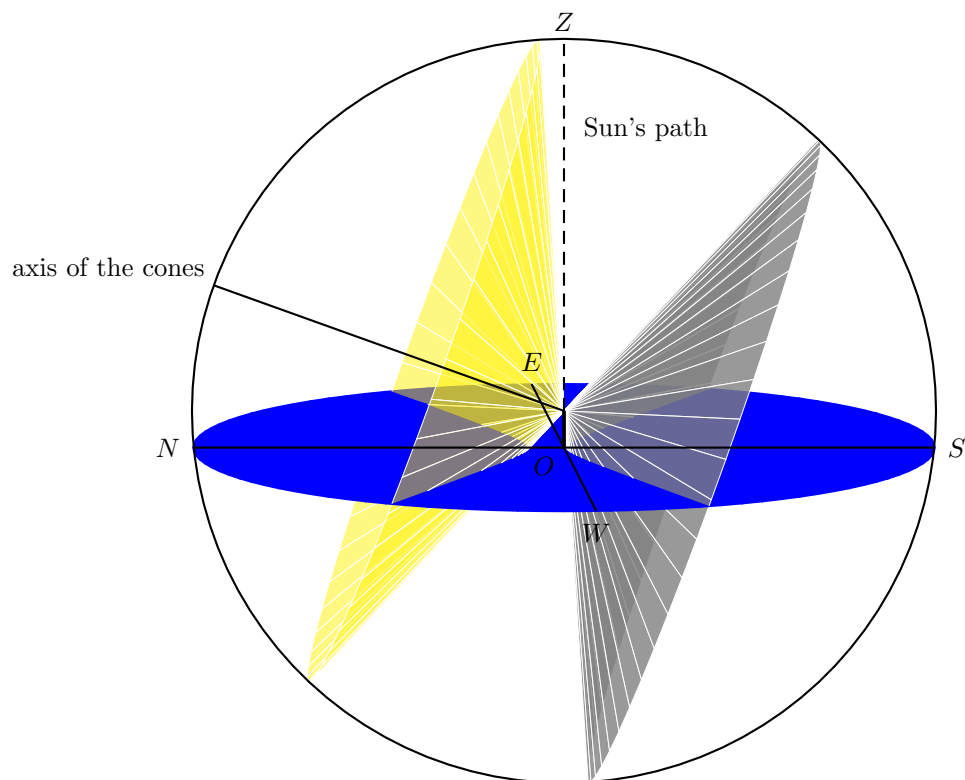


Figure 5.2: Figure 1.2 drawn using Sketch

Chapter 6

Conclusion

After trying out the various programs, we can conclude that each has its strengths and weaknesses. Our conclusions are based on our two months experience of using the programs to draw certain 3D pictures. Experienced users could probably overcome many of the limitations we describe, but we hope that other newbies may find our observations useful.

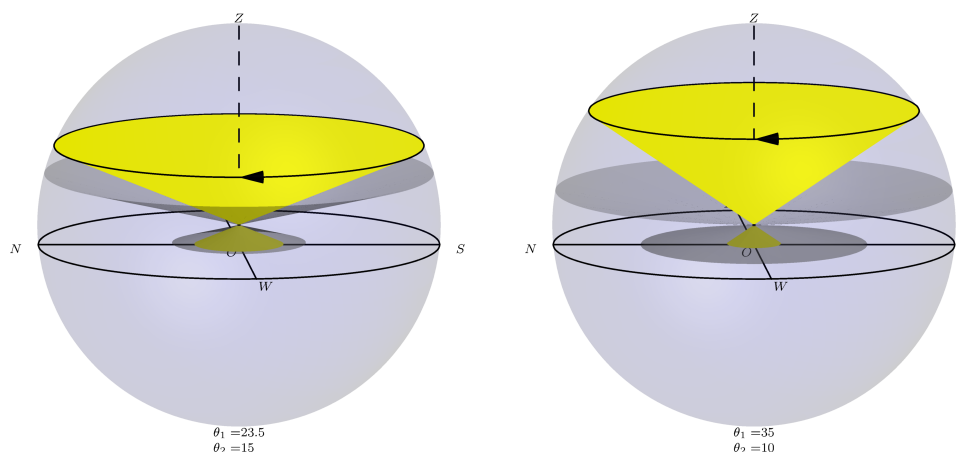


Figure 6.1: Changing the height of cones using Asymptote

6.1 MetaPost

MetaPost is inherently 2D. That means that you cannot right away use it for 3D drawings. You will have to design the objects on your own. This is definitely possible for experienced MetaPost users, as seen in the paper of Roegel [1] but probably not the way to go for newbies who are not already experienced MetaPost users. Although there are 3D packages available online,

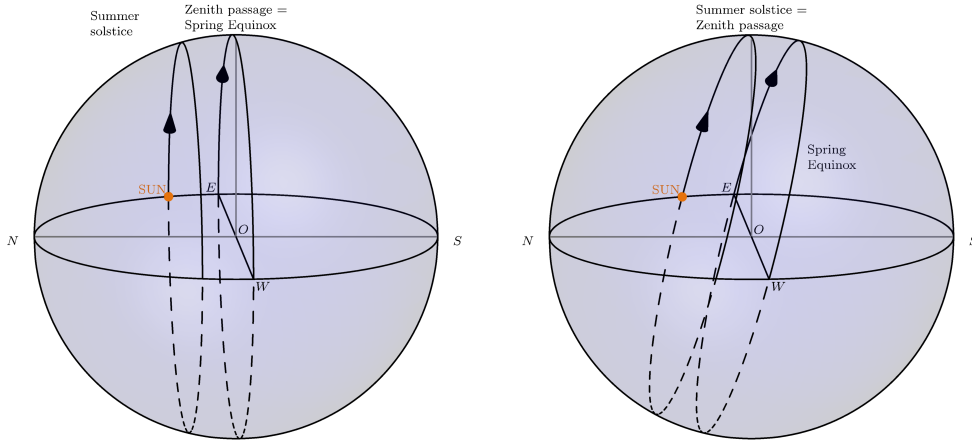


Figure 6.2: Recycling code for similar diagrams (Asymptote)

they are still “under construction”. According to a review of 3D MetaPost packages by L. N. Gonçalves [12], m3D is the best 3D MetaPost package available. We have tried m3D but found it to be difficult to use.

One good point about MetaPost is its `intersectionpoint` operator. Although it is meant for finding intersections of 2D curves, it is still very useful. We can find the intersection points after projecting 3D objects onto a plane.

6.2 Asymptote

We find Asymptote to be a very impressive and powerful graphics package. The Asymptote website [6] claims that “a major advantage of Asymptote over other graphics packages is that it is a programming language, as opposed to just a graphics program”. Based on our experience, this claim is justified.

Asymptote is much stronger than the other three programs in terms of calculation. It follows the IEEE Standard for Floating-Point Arithmetic, which can compute values to an accuracy of 16 digits. For instance, we can calculate the normal vector of a circle in 3D that we drew using Asymptote. We can use Asymptote to “help” PSTricks do some computations like calculating the normal vector of a circle and points of intersection. It is also possible to customize new features using Asymptote’s own C++-like graphics programming language. Asymptote is also the only program among the four that can produce rotatable figures (PRC format).

During our research, we tried to use Asymptote to draw the intersection curve between a cone and a horizontal plane. We obtained a picture with jagged edges (Fig. 6.3). We posted a question on the Asymptote forum, but

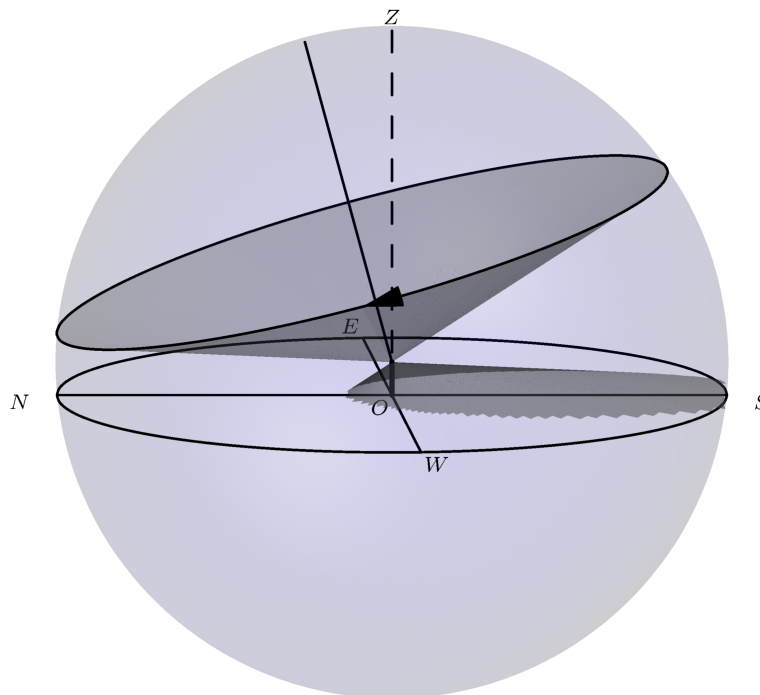


Figure 6.3: Base of cone has jagged edges

there was no response. This seems to be one weakness of Asymptote, since `pst-solides3d` can do this easily.

Lastly, compared to other three programs, Asymptote requires much more computer memory. A quick search on the Asymptote Forum revealed that many people face “out of memory” errors while using Asymptote. The developers are aware of this issue though, and have come up with measures to alleviate the problem. (The latest stable release, Asymptote 2.00, is faster and uses less memory than previous versions.)

Asymptote is rapidly evolving and improving. During the course of this research alone, Asymptote has been upgraded from version 1.94 to version 2.01.

6.3 PSTricks

A major strength of the `pst-solides3d` package is its ability to draw curves of intersection between a solid and a plane. Notably, it is not easy for Asymptote to draw curves of intersection.

Also, `pst-solides3d` is able to do automatic dashing of hidden lines. This is useful when drawing spheres and their components.

The `pst-solides3d` package is good at drawing different kinds of solids. However, it cannot calculate points of intersection in 3D. One possible workaround is to use another program to calculate the points of intersection, or calculate it manually.

6.4 Sketch

The current version of Sketch has no built-in definition of curves or a Bézier curve, but the next version will have a three-dimensional equivalent of the PSTricks command `\pscurve`.

Sketch is more suited to the drawing of polyhedra. In the Sketch documentation, it is stated that the “kind of problem Sketch was meant to solve” is to draw “a polygonal approximation of a truncated cone” [9].

Other than producing PSTricks output, Sketch can also produce TikZ/PGF output.

Like `pst-solides3d`, Sketch is unable to calculate points of intersection.

6.5 Comparison

	Metapost	Asymptote	PSTricks	Sketch
Inherently 3D?	No	Yes	Yes	Yes
Automatic dashing of hidden lines	No	Yes	Yes	No
Calculate points of intersection	Yes (2D)	Yes (3D)	No	No
Draw curve of intersection between solid and plane	No	No	Yes	No
Output is rotatable	No	Yes	No	No

We found Sketch to be too limited for our purposes and we did not have time to code our own 3D objects in MetaPost. We liked both Asymptote and PSTricks. However, neither of them were able to do everything we needed. However, with some minor external computations, PSTricks was able to draw everything. Asymptote also came close, but was not able to draw the curves of intersection.

We found the Asymptote syntax easy to learn and similar to other programming languages. Unless you have PostScript experience, the PSTricks syntax may seem harder to learn.

However, we strongly recommend both programs, and encourage you to explore the many websites with examples.

Bibliography

- [1] Denis Roegel, Spheres, Great Circles and Parallels, TUGboat, Vol. 30 (2009), No. 1, pg. 80–87, <http://www.loria.fr/~roegel/TeX/tb94roegel-spheres.pdf> . 1, 2, 2.2, 2.3, 2.8, 2.9, 6.1
- [2] MetaPost, Wikipedia, the free encyclopedia, <http://en.wikipedia.org/wiki/MetaPost> . 2
- [3] John D. Hobby, MetaPost manual, <http://www.tug.org/docs/metapost/mpman.pdf> . 3, 2.1, 2.9
- [4] Asymptote, Wikipedia, the free encyclopedia, [http://en.wikipedia.org/wiki/Asymptote_\(vector_graphics_language\)](http://en.wikipedia.org/wiki/Asymptote_(vector_graphics_language)) . 3
- [5] Philippe Ivaldi, Asymptote, <http://www.piprime.fr/developpeur/asymptote/> . 3.5, 3.9
- [6] Asymptote Website, <http://asymptote.sourceforge.net/> . 1, 3.8, 6.2
- [7] PSTricks, Wikipedia, the free encyclopedia, <http://en.wikipedia.org/wiki/PSTricks> . 4
- [8] Jean-Paul Vignault, Manuel Luque, Arnaud Schmittbuhl, pst-solides3d documentation (English), <http://tug.ctan.org/tex-archive/graphics/pstricks/contrib/pst-solides3d/pst-solides3d-doc.pdf> . 4, 4.3, 4.4, 4.8, 4.9
- [9] Eugene Ressler, Sketch documentation, <http://www.frontiernet.net/~eugene.ressler/sketch.html> . 5, 5.3, 5.8, 5.9, 6.4
- [10] LEE Suling, SIM Xiu Juan, YANG Yunhui, Measuring the Tropical Year in Chinese Astronomy, Honours Thesis, National University of Singapore, Supervisor: Helmer Aslaksen, 2006, <http://www.math.nus.edu.sg/aslaksen/projects/LSY.pdf> . 1.2

- [11] Anthony Phan, m3D, <http://www-math.univ-poitiers.fr/~phan/m3Dplain.html> . 2.8
- [12] L. N. Gonçalves, FEATPOST and a Review of 3D Meta-Post Packages, TUG 2004 preprints, (2004), pg. 99–104, <http://www.ctan.org/tex-archive/graphics/metapost/contrib/macros/featpost/doc/tug2004.pdf> . 6.1
- [13] Asymptote: Advanced Configuration, http://www.artofproblemsolving.com/Wiki/index.php/Asymptote:_Advanced_Configuration . 2
- [14] Timothy Van Zandt, PSTricks Manual, <ftp://mirror.upm.edu.my/ctan/graphics/pstricks/base/doc/pstricks-doc.pdf> . 4.1, 3
- [15] Michel Goossens, Frank Mittelbach, Sebastian Rahtz, Denis Roegel, Herbert Voss, The L^AT_EX Graphics Companion, (2nd Edition) .
- [16] Herbert Voss, PSTricks: Graphics and PostScript for T_EX and L^AT_EX .

Acknowledgements

First and foremost I would like to thank my supervisor Associate Professor Helmer Aslaksen, who granted me an opportunity to do this project under him. Before embarking on this project, I had totally zero knowledge about graphics programming. I would like to thank him for his invaluable guidance, and for his patience in teaching me, enabling me to develop a clearer understanding of the subject. I have truly learnt a lot about mathematics, astronomy, computer graphics, and many other useful research skills.

I am also thankful to my parents, who sponsored my school fees for the Special Term and provided me with much needed moral support.

Finally, I must thank God, who “made the Pleiades and Orion . . . turns the shadow of death into morning and makes the day dark as night” (Amos 5:8).